2026-6-9 哈希表封装myunorder_map和myunorder_set

- 1. 第一层：哈希表的封装
  - 1.1 复用哈希表框架
  - 1.2 支持insert函数
  - 1.3 在底层实现迭代器
    - 1.3.1 定义迭代器结构
    - 1.3.2 在哈希表中定义begin和end
    - 1.3.1 bengin和end的上层封装
- 2. 第二层：上层实现unorder_map和unorder_set
  - 2.1 unorder_map实现operater[]
  - 2.2 unorder_map的具体实现
  - 2.3 unorder_set的具体实现
- 3. 第三层：上层运用

1. 第一层：哈希表的封装

1.1 复用哈希表框架

解决数据不能取模的问题：

```
template<class K>
struct HashFunc
{
    size_t operator()(const K& key)
    {
        return (size_t)key;
    }
};
template<>
struct HashFunc<string>
{
    // BKDR
    size_t operator()(const string& str)
    {
        size_t hash = 0;
        for (auto ch : str)
        {
            hash += ch;
            hash *= 131;
        }
```

```
        return hash;
    }
};
```

质数表

```
inline unsigned long __stl_next_prime(unsigned long n)

{

    // Note: assumes long is at least 32 bits.

    static const int __stl_num_primes = 28;

    static const unsigned long __stl_prime_list[__stl_num_primes] =

    {

        53, 97, 193, 389, 769,

        1543, 3079, 6151, 12289, 24593,

        49157, 98317, 196613, 393241, 786433,

        1572869, 3145739, 6291469, 12582917, 25165843,

        50331653, 100663319, 201326611, 402653189, 805306457,

        1610612741, 3221225473, 4294967291

    };

    const unsigned long* first = __stl_prime_list;

    const unsigned long* last = __stl_prime_list + __stl_num_primes;

    const unsigned long* pos = lower_bound(first, last, n);

    return pos == last ? *(last - 1) : *pos;

}
```

定义哈希表节点结构：

```
template<class T>

struct HashNode

{

    T _data;

    HashNode<T>* _next;

    HashNode(const T& data)

        :_data(data)

        ,_next(nullptr)

    {}

};
```

对哈希桶结构进行复用：

```
template<class K, class T, class KeyOfT, class Hash = HashFunc<K>>

class HashTable

{

    typedef HashNode<T> Node;

public:

    HashTable()

        :_tables(__stl_next_prime(1), nullptr)

        ,_n(0)

    {}
    private:

    //vector<list<pair<K, V>>> _tables;

    vector<Node*> _tables;
```

```
    size_t _n = 0;  // 实际存储的数据个数
};
```

析构函数：

```
~HashTable()
    {
        for (size_t i = 0; i < _tables.size(); i++)
        {
            Node* cur = _tables[i];
            // 当前桶的节点重新映射挂到新表
            while (cur)
            {
                Node* next = cur->_next;
                delete cur;
                cur = next;
            }
            _tables[i] = nullptr;
        }
    }
```

1.2 支持insert函数

```
bool Insert(const T& data)
    {
```

```
        KeyOfT kot;
        if (Find(kot(data)))
            return false;
        Hash hs;
        // 负载因子==1扩容
        if (_n == _tables.size())
        {
            //HashTable<K, V> newHT;
            //newHT._tables.resize(_tables.size()*2);
            //// 遍历旧表将所有值映射到新表
            //for (auto cur : _tables)
            //{
            //  while (cur)
            //  {
            //      newHT.Insert(cur->_kv);
            //      cur = cur->_next;
            //  }
            //}
            //_tables.swap(newHT._tables);
            vector<Node*> newtables(__stl_next_prime(_tables.size()+1));
            for (size_t i = 0; i < _tables.size(); i++)
            {
                Node* cur = _tables[i];
```

```
                // 当前桶的节点重新映射挂到新表
                while (cur)
                {
                    Node* next = cur->_next;
                    // 插入到新表
                    size_t hashi = hs(kot(cur->_data)) % newtables.size();
                    cur->_next = newtables[hashi];
                    newtables[hashi] = cur;
                    cur = next;
                }
                _tables[i] = nullptr;
            }
            _tables.swap(newtables);
        }
        size_t hashi = hs(kot(data)) % _tables.size();
        // 头插
        Node* newNode = new Node(data);
        newNode->_next = _tables[hashi];
        _tables[hashi] = newNode;
        ++_n;
        return true;
    }
```

1.3 在底层实现迭代器

1.3.1 定义迭代器结构

```
// 前置声明

template<class K, class T, class KeyOfT, class Hash>

class HashTable;

template<class K, class T, class Ref, class Ptr, class KeyOfT, class Hash>

struct HTIterator

{

    typedef HashNode<T> Node;

    typedef HashTable<K, T, KeyOfT, Hash> HT;

    typedef HTIterator<K, T, Ref, Ptr, KeyOfT, Hash> Self;

    Node* _node;

    const HT* _ht;

    HTIterator(Node* node, const HT* ht)

        :_node(node)

        , _ht(ht)

    {}

    Ref operator*()

    {

        return _node->_data;

    }

    Ptr operator->()

    {

        return &_node->_data;
```

```
    }

    Self& operator++()

    {

        if (_node->_next)  // 当前还有节点

        {

            _node = _node->_next;

        }

        else  // 当前桶为空，找下一个不为空的桶的第一个

        {

            size_t hashi = Hash()(KeyOfT()(_node->_data)) % _ht-
>_tables.size();

            ++hashi;

            while (hashi != _ht->_tables.size())

            {

                if (_ht->_tables[hashi])

                {

                    _node = _ht->_tables[hashi];

                    break;

                }

                hashi++;

            }

            // 最后一个桶的最后一个节点已经遍历结束，走到end()去，nullptr充当end()

            if (hashi == _ht->_tables.size())

            {
```

```
                _node = nullptr;
            }
        }
        return *this;
    }
    bool operator!=(const Self& s) const
    {
        return _node != s._node;
    }
    bool operator==(const Self& s) const
    {
        return _node == s._node;
    }
};
```

1.3.2 在哈希表中定义begin和end

1.3.1 bengin和end的上层封装

2. 第二层：上层实现unorder_map和unorder_set

2.1 unorder_map实现operater[]

2.2 unorder_map的具体实现

```
#pragma once
#include"HashTable.h"
```

```
namespace bit
{
    template<class K, class V>
    class unordered_map
    {
        struct MapKeyOfT
        {
            const K& operator()(const pair<K, V>& kv)
            {
                return kv.first;
            }
        };
    public:
        bool insert(const pair<K, V>& kv)
        {
            return _t.Insert(kv);
        }
    private:
        HashTable<K, pair<K, V>, MapKeyOfT> _t;
    };
    void test_umap()
    {
        unordered_map<string, string> dict;
```

```
        dict.insert({ "insert", "" });
        dict.insert({ "sort", "" });
        dict.insert({ "test", "" });
        /*for (auto& [k, v] : dict)
        {
            cout << k << ":" << v << endl;
        }*/
    }
};
```

2.3 unorder_set的具体实现

```
#pragma once
#include"HashTable.h"
namespace bit
{
    template<class K>
    class unordered_set
    {
        struct SetKeyOfT
        {
            const K& operator()(const K& key)
            {
                return key;
```

```
        }
    };
public:
    bool insert(const K& k)
    {
        return _t.Insert(k);
    }
private:
    HashTable<K, K, SetKeyOfT> _t;
};
void test_uset()
{
    unordered_set<int> s1;
    s1.insert(45);
    s1.insert(5);
    s1.insert(13);
    s1.insert(45);
    /*auto it1 = s1.begin();
    while (it1 != s1.end())
    {
        cout << *it1 << " ";
        ++it1;
    }
```

```
        cout << endl;*/
    }
};
```

3. 第三层：上层运用

```
#define _CRT_SECURE_NO_WARNINGS 1
#include<iostream>
#include<string>
#include<set>
#include<unordered_set>
#include<unordered_map>
using namespace std;
void test_u_set_map()
{
    unordered_set<int> s1 = {2,3,5,6,7,2};
    s1.insert(45);
    auto it1 = s1.begin();
    while (it1 != s1.end())
    {
        cout << *it1 << " ";
        ++it1;
    }
    cout << endl;
```

```
    unordered_map<string, string> dict;
    dict.insert({ "insert", "插入" });
    dict.insert({ "sort", "排序" });
    dict.insert({ "test", "测试" });
    for (auto& [k, v] : dict)
    {
        cout << k << ":" << v << endl;
    }
}
int test_op()
{
    const size_t N = 1000000;
    unordered_set<int> us;
    set<int> s;
    vector<int> v;
    v.reserve(N);
    srand(time(0));
    for (size_t i = 0; i < N; ++i)
    {
        //v.push_back(rand()); // N比较大时，重复值比较多
        v.push_back(rand() + i); // 重复值相对少
        //v.push_back(i); // 没有重复，有序
    }
```

```
    size_t begin1 = clock();
    for (auto e : v)
    {
        s.insert(e);
    }
    size_t end1 = clock();
    cout << "set insert:" << end1 - begin1 << endl;
    size_t begin2 = clock();
    us.reserve(N);
    for (auto e : v)
    {
        us.insert(e);
    }
    size_t end2 = clock();
    cout << "unordered_set insert:" << end2 - begin2 << endl;
    int m1 = 0;
    size_t begin3 = clock();
    for (auto e : v)
    {
        auto ret = s.find(e);
        if (ret != s.end())
        {
            ++m1;
```

```
        }
    }
    size_t end3 = clock();
    cout << "set find:" << end3 - begin3 << "->" << m1 << endl;
    int m2 = 0;
    size_t begin4 = clock();
    for (auto e : v)
    {
        auto ret = us.find(e);
        if (ret != us.end())
        {
            ++m2;
        }
    }
    size_t end4 = clock();
    cout << "unorered_set find:" << end4 - begin4 << "->" << m2 << endl;
    cout << "插入数据个数：" << s.size() << endl;
    cout << "插入数据个数：" << us.size() << endl << endl;
    size_t begin5 = clock();
    for (auto e : v)
    {
        s.erase(e);
    }
```

```
    size_t end5 = clock();
    cout << "set erase:" << end5 - begin5 << endl;
    size_t begin6 = clock();
    for (auto e : v)
    {
        us.erase(e);
    }
    size_t end6 = clock();
    cout << "unordered_set erase:" << end6 - begin6 << endl << endl;
    return 0;
}
#include"HashTable.h"
#include"UnorderedSet.h"
#include"UnorderedMap.h"
int main()
{
    //test_u_set_map();
    //test_op();
    //hash_bucket::TestHT1();
    //TestHT2();
    //unordered_set<int> s;
    bit::test_uset();
    bit::test_umap();
```

```
    return 0;

}
```